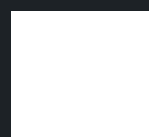




Guia completo Entrevistas

Perguntas feitas em vagas JavaScript, React e React Native

BY SUJEITO PROGRAMADOR





Separamos um Guia completo com perguntas que mais são feitas em entrevistas

JavaScript, React JS e React Native

Sumário

1. JavaScript	1
2. React JS	7
3. React Native	15

Perguntas JavaScript



1) O que é JavaScript e como ele difere de outras linguagens de programação?

R: JavaScript é uma linguagem de programação de alto nível, interpretada e baseada em eventos, com suporte para manipulação do DOM (Document Object Model). Ao contrário de linguagens como Java, que são compiladas, o JavaScript é executado diretamente no navegador, o que o torna ideal para desenvolvimento web interativo.

2) Qual é a diferença entre var, let e const?

- **var:** Declara variáveis que são hoisted (elevadas ao topo) e tem escopo de função, o que pode levar a comportamentos inesperados.
- **let:** Introduzido no ES6, tem escopo de bloco e não é hoisted, sendo mais seguro que var em muitos casos.
- **const:** Também tem escopo de bloco, mas as variáveis declaradas com const não podem ser reatribuídas

3) O que é hoisting em JavaScript?

Hoisting é o comportamento do JavaScript em mover as declarações de variáveis e funções para o topo de seu escopo antes de executar o código. Isso significa que você pode usar uma variável antes de declará-la, mas apenas para variáveis declaradas com var



4) O que é o closure e por que ele é importante?

Um closure é uma função que "lembra" o ambiente em que foi criada. Ele permite que uma função interna acesse variáveis da função externa mesmo depois que a função externa já tenha sido executada. Isso é útil para manter estados privados.

```
function criaUser() {  
  const name = "Sujeito Programador";  
  
  function mostraNome() {  
    console.log(name);  
  }  
  return mostraNome;  
}  
  
const myFunc = criaUser();  
myFunc();
```

5) Explique o que são promises e como elas funcionam

Promises são objetos que representam a eventual conclusão (ou falha) de uma operação assíncrona. Elas têm três estados: pending (pendente), fulfilled (realizada) e rejected (rejeitada). As promises permitem um código assíncrono mais legível, evitando o "callback hell".

6) Qual é a diferença entre == e === ?

- == Compara apenas os valores, realizando conversões implícitas de tipos (coerção).
- === Compara tanto o valor quanto o tipo, sem conversões implícitas, sendo a forma mais segura de comparação.



7) Qual a diferença entre undefined e null?

- undefined: Uma variável é declarada mas não inicializada.
- null: Um valor explicitamente atribuído, representando a ausência de qualquer valor.

8) O que é um callback no JavaScript?

Em JavaScript, as funções são objetos que podem receber diferentes funções como argumentos e serem retornadas por outras funções. Um callback é uma função JavaScript passada para outra função como um argumento ou parâmetro. Essa função é executada quando a função à qual ela foi passada é executada.

9) O que são laços e iterações em JavaScript?

São formas de realizar operações múltiplas vezes. Os tipos de laços e iterações do JavaScript mais comuns são: while, do while, for, for in



10) Explique o uso de async e await ?

async e await são palavras-chave introduzidas no ES2017 que facilitam o trabalho com operações assíncronas. async marca uma função como assíncrona e await é usada para "esperar" a resolução de uma promise, permitindo escrever código assíncrono de maneira mais síncrona.

11) O que é o objeto this em JavaScript?

this refere-se ao contexto no qual a função é chamada. Seu valor depende de onde e como a função é invocada. Em métodos de objetos, this geralmente aponta para o próprio objeto

12) O que são arrow functions e como elas diferem de funções normais?

As arrow functions introduzidas no ES6 têm uma sintaxe mais curta e não criam seu próprio contexto this, herdando-o do escopo léxico onde foram definidas. Elas são usadas principalmente para simplificar o código e evitar confusões com o this.

```
let minhaFuncao = (a, b) => a * b;  
hello = () => "Olá Mundo"
```

13) O que é destructuring em JavaScript?

Destructuring é uma maneira de extrair valores de arrays ou objetos de forma concisa. Por exemplo:

```
const { name, age } = person;  
//Isso atribui as propriedades name e age do objeto  
person diretamente às variáveis.
```



14) Explique o que é a event loop no JavaScript.

O event loop é o mecanismo que permite o JavaScript realizar operações não-bloqueantes, apesar de ser single-threaded. Ele processa eventos e callbacks enfileirados, permitindo que o JavaScript execute código assíncrono, como requisições de rede ou timers, sem bloquear o fluxo principal.

15) O que é o JSON e como ele é usado no JavaScript?

JSON (JavaScript Object Notation) é um formato leve para troca de dados, muito utilizado para comunicação entre um cliente e servidor. No JavaScript, você pode converter objetos para JSON com `JSON.stringify()` e transformar JSON em objetos com `JSON.parse()`.

16) O que é o DOM (Document Object Model) e como o JavaScript interage com ele?

O DOM é uma representação em árvore dos elementos de uma página web. Com o JavaScript, é possível acessar, manipular e alterar o DOM dinamicamente, permitindo que você mude o conteúdo, estrutura e estilo de uma página sem recarregá-la. Isso é feito através de métodos como `document.getElementById()`, `document.querySelector()`, e propriedades como `innerHTML`.

17) O que são cookies, localStorage e sessionStorage e como diferem?

- **Cookies:** Pequenos pedaços de dados armazenados no navegador, enviados ao servidor com cada requisição HTTP. Eles têm uma capacidade limitada (aproximadamente 4KB) e são usados principalmente para gerenciamento de sessões e autenticação.
- **localStorage:** Armazena dados no navegador sem data de expiração, ou seja, os dados persistem mesmo após o fechamento do navegador. Ideal para salvar dados que precisam estar disponíveis de forma permanente.
- **sessionStorage:** Similar ao localStorage, mas os dados são removidos assim que a aba ou janela do navegador é fechada.

Perguntas React JS



1) O que é o React e quais são suas principais características?

React é uma biblioteca JavaScript de código aberto desenvolvida pelo Facebook para a construção de interfaces de usuário, principalmente em aplicações de página única (SPAs). Suas principais características incluem:

- Componentes: A UI é dividida em componentes reutilizáveis.
- Virtual DOM: Em vez de atualizar o DOM diretamente, o React usa um Virtual DOM que aplica mudanças de forma mais eficiente.
- Unidirectional Data Flow: O fluxo de dados no React segue uma única direção, facilitando o controle e a depuração do estado.

2) O que é JSX e por que é usado no React?

JSX (JavaScript XML) é uma extensão de sintaxe do JavaScript que permite escrever HTML diretamente dentro do JavaScript. Ele facilita a criação de componentes no React ao integrar a estrutura da interface com a lógica JavaScript, tornando o código mais legível e declarativo.



3) Qual é a diferença entre componentes de classe e componentes funcionais no React?

- Componentes de Classe: Eram a forma tradicional de criar componentes, usando classes ES6. Eles permitem o uso de estado e métodos de ciclo de vida como `componentDidMount()` e `componentWillUnmount()`.
- Componentes Funcionais: Introduzidos mais recentemente (com Hooks no React 16.8), são funções simples que retornam JSX. Com os Hooks (`useState`, `useEffect`), os componentes funcionais podem manipular estado e ciclo de vida, substituindo as classes em grande parte.

4) O que são Hooks no React e por que eles são importantes?

Hooks são funções que permitem o uso de estado e outros recursos do React em componentes funcionais. Eles são importantes porque:

- Tornam os componentes funcionais mais poderosos e flexíveis.
- Eliminam a necessidade de usar componentes de classe.
- Facilitam a reutilização de lógica de estado entre diferentes componentes.

Os Hooks mais comuns incluem `useState`, `useEffect`, `useContext`, `useMemo` e `useCallback`.

5) Explique o que é o Hook `useState` e como ele funciona ?

`useState` é um Hook que permite adicionar estado a componentes funcionais. Ele retorna um array com dois valores: o estado atual e uma função para atualizá-lo. Exemplo:

```
const [count, setCount] = useState(0);

//Aqui, count é o valor do estado inicial (0) e setCount
é a função usada para atualizá-lo.
```



6) O que é o Hook `useEffect` e quais são suas aplicações?

- Componentes de Classe: Eram a forma tradicional de criar componentes, usando classes ES6. Eles permitem o uso de estado e métodos de ciclo de vida como `componentDidMount()` e `componentWillUnmount()`.
- Componentes Funcionais: Introduzidos mais recentemente (com Hooks no React 16.8), são funções simples que retornam JSX. Com os Hooks (`useState`, `useEffect`), os componentes funcionais podem manipular estado e ciclo de vida, substituindo as classes em grande parte.

7) O que é o Hook `useEffect` e quais são suas aplicações?

`useEffect` é usado para realizar efeitos colaterais em componentes funcionais, como chamadas de API, assinaturas de eventos e manipulação direta do DOM. Ele pode ser configurado para rodar após cada renderização, apenas uma vez, ou quando determinadas dependências mudarem. Exemplo:

8) O que é o Virtual DOM e por que ele é utilizado no React?

O Virtual DOM é uma representação leve do DOM real que o React mantém na memória. Quando o estado de um componente muda, o Virtual DOM é atualizado primeiro, e então comparado com a versão anterior (processo chamado de reconciliation). Apenas as partes que realmente mudaram no DOM real são atualizadas, otimizando o desempenho.



9) Como o React trata o gerenciamento de estado em aplicações mais complexas?

Para gerenciar estados complexos, o React pode usar bibliotecas externas como:

- Redux: Uma solução popular que centraliza o estado global da aplicação em uma store, permitindo a comunicação entre componentes distantes.
- Context API: Oferece uma maneira nativa de compartilhar estado entre componentes sem precisar passar props manualmente em cada nível.
- Zustand ou Recoil: Outras bibliotecas que são alternativas mais simples ou mais flexíveis para gerenciar estado global.

10) Explique o conceito de prop drilling e como evitá-lo ?

Prop drilling acontece quando propriedades precisam ser passadas manualmente através de vários níveis de componentes intermediários apenas para chegar a um componente mais profundo. Para evitá-lo, podemos usar a Context API ou bibliotecas de gerenciamento de estado global, como Redux ou Zustand.

11) Qual a função do key em listas renderizadas no React?

A propriedade key é usada para identificar de forma única os itens de uma lista renderizada no React. Isso ajuda o React a otimizar a renderização, identificando quais itens mudaram, foram adicionados ou removidos, minimizando operações desnecessárias no DOM.



12) O que são controlled e uncontrolled components no React ?

- **Controlled Components:** Componentes cujos valores de formulário são controlados pelo estado do React. As alterações no valor do formulário são feitas atualizando o estado.
- **Uncontrolled Components:** Componentes onde os valores do formulário são controlados diretamente pelo DOM, usando refs para acessar os valores, sem interferência no estado React.

13) O que é o React Router e para que ele serve?

O React Router é uma biblioteca de roteamento usada para gerenciar a navegação em aplicativos React de página única (SPA). Ele permite definir rotas, redirecionar usuários, passar parâmetros e controlar a renderização de componentes com base na URL.

14) O que são Higher-Order Components (HOC) e como eles são utilizados?

Higher-Order Components (HOCs) são funções que recebem um componente como argumento e retornam um novo componente aprimorado. Eles são usados para reutilizar lógica entre componentes. Exemplo: autenticação, autorização ou manipulação de dados podem ser implementados como HOCs.

Vamos ver na próxima página um exemplo Componente HOC:



Exemplo Higher-Order Components (HOC)

```
import React, { useState } from 'react';

// Este é o nosso HOC
function withCounter(WrappedComponent) {
  return function(props) {
    const [count, setCount] = useState(0);

    const incrementCount = () => {
      setCount(count + 1);
    };

    return (
      <WrappedComponent
        count={count}
        incrementCount={incrementCount}
        {...props}
      />
    );
  };
}

// Um componente que você pode passar para o nosso HOC
function ClickCounter({ count, incrementCount }) {
  return (
    <div>
      <button onClick={incrementCount}>
        Clicked {count} times
      </button>
    </div>
  );
}

// Aplicando o HOC
const EnhancedClickCounter = withCounter(ClickCounter);

function App() {
  return (
    <div>
      <EnhancedClickCounter />
    </div>
  );
}

export default App;
```

- **withCounter** é o HOC que adiciona a funcionalidade de contagem ao componente que ele envolve.
- **ClickCounter** é um componente simples que exibe um botão e o número de vezes que foi clicado.
- **EnhancedClickCounter** é o novo componente criado pelo HOC, que inclui a lógica de contagem.



15) O que é o Suspense e o React.lazy e como eles são usados?

- Suspense e React.lazy são usados para carregamento dinâmico de componentes. React.lazy permite que você importe componentes sob demanda, enquanto Suspense permite mostrar um fallback (como um spinner) até que o componente seja carregado.

16) Qual a importância do Hook useContext no React

useContext permite que um componente acesse o valor de um contexto React sem precisar passar props manualmente. Ele simplifica a comunicação entre componentes em diferentes níveis da árvore de componentes.

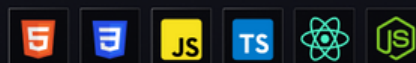
17) O que são props no React ?

Props em React significa propriedades. Elas funcionam como um canal de comunicação do pai para o filho.

18) O que é o Redux?

Redux é uma biblioteca JavaScript de código aberto, ele funciona como um container para aplicações JavaScript e é usado para gerenciar o estado das aplicações de forma global evitando prop drilling e trazendo mais performance e evitando renderizações desnecessárias.

Full Stack PRO



Aprenda do zero a criar sites, sistemas web completos, SaaS passo a passo.

[Clique aqui para acessar](#)



19) Qual a principal diferença entre Props e States?

O estado é mutável, enquanto as Props são imutáveis. Isso significa que o estado é interno e gerenciado pelo componente, enquanto as props são externos e gerenciados por qualquer coisa que renderize o componente.

20) O que são Error Boundaries em React e como eles funcionam?

Error Boundaries são componentes React que capturam erros de JavaScript em componentes filhos durante a renderização, no ciclo de vida ou em construtores, e exibem uma interface de fallback em vez de quebrar a aplicação.

Ou seja eles são usados para impedir que erros não tratados causem falhas completas na UI (interface).

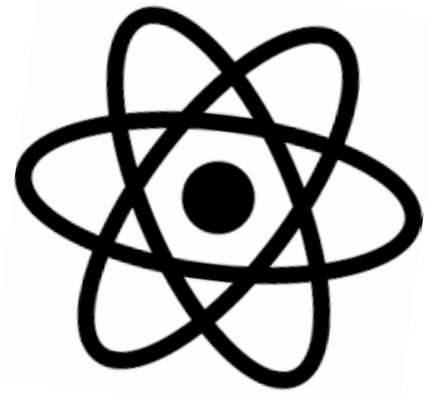
21) O que são Fragments?

É um padrão ou prática comum no React para um componente retornar vários elementos.

Fragments permitem que você agrupe uma lista de filhos sem adicionar nós extras ao DOM sem usar uma tag que modifique ou cause alterações na UI. Representado por: `<> </>` ou `<Fragment> </Fragment>`

```
function Filme({ title, description, date }) {  
  return (  
    <>  
      <h2>{title}</h2>  
      <p>{description}</p>  
      <p>{date}</p>  
    </>  
  );  
}
```

Perguntas React Native

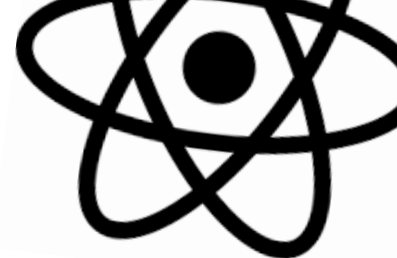


1) O que é o React Native e como ele difere do React JS?

React Native é um framework para construir aplicações móveis nativas usando JavaScript e React. Ao contrário do React JS, que é focado em web, o React Native compila os componentes React para componentes nativos que são renderizados diretamente no Android e iOS. React Native utiliza APIs nativas para funções de hardware e sistema, enquanto o React JS usa o DOM do navegador.

2) O que são native modules no React Native e por que eles são importantes?

Native modules são partes de código escritas em linguagens nativas, como Java, Swift, ou Objective-C, que interagem com a aplicação React Native. Eles são importantes quando você precisa acessar funcionalidades específicas do dispositivo que não são expostas pelo React Native de forma nativa, como sensores, Bluetooth, ou bibliotecas específicas da plataforma.



3) Como funciona a navegação em React Native e quais são as bibliotecas mais usadas?

- A navegação em React Native é feita de forma diferente da web, usando bibliotecas específicas para navegação em dispositivos móveis. As bibliotecas mais usadas são:
- React Navigation: A mais popular, oferece navegação em pilha, abas, gavetas, etc.
- Expo Router (Expo)

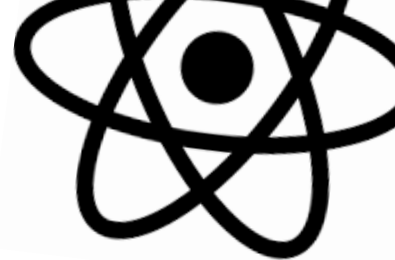
4) O que é o Flexbox no React Native e como ele ajuda no layout?

O Flexbox é um sistema de layout usado para organizar elementos de forma responsiva em React Native.

Ele é baseado em linhas e colunas, e facilita a criação de layouts adaptáveis para diferentes tamanhos de tela. Propriedades como `flexDirection`, `justifyContent`, e `alignItems` são usadas para controlar o posicionamento de elementos no layout.

5) Qual é a função do bridge no React Native?

O bridge é a camada que permite que o código JavaScript e o código nativo se comuniquem no React Native. Ele permite a troca de informações entre a camada JavaScript e o código nativo (escrito em Swift, Objective-C, Java ou Kotlin), possibilitando o uso de funcionalidades nativas do dispositivo.



6) O que é o Metro bundler no React Native?

O Metro Bundler é o empacotador padrão usado pelo React Native para processar e otimizar os arquivos JavaScript. Ele transforma o código JavaScript e JSX em um bundle que pode ser lido e executado nativamente no aplicativo.

Ele também faz hot reloading e live reloading durante o desenvolvimento, facilitando o processo de desenvolvimento.

7) O que é Expo ?

Expo é um Framework, basicamente conjunto de ferramentas e serviços que simplificam o desenvolvimento com React Native, fornecendo uma camada de abstração sobre as configurações nativas.

O expo já trás vários componentes e integração criada para facilitar o desenvolvimento dos Apps com React Native como componente camera, localização até facilitando deploy para gerar o build.

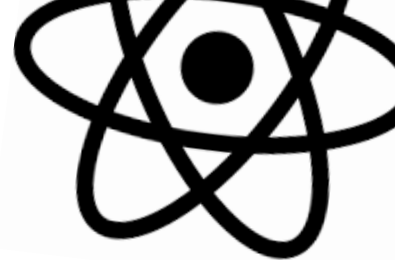
8) Qual a diferença entre ScrollView e FlatList em React Native?

- ScrollView: É usado para renderizar todos os elementos filhos de uma só vez, o que pode ser problemático em listas grandes, pois afeta o desempenho.
- FlatList: É otimizado para listas grandes e só renderiza os itens visíveis na tela, carregando mais itens sob demanda à medida que o usuário rola a lista, tornando-o mais eficiente em termos de memória e desempenho.

Full Stack PRO

Aprenda do zero a criar sites, sistemas web completos, SaaS passo a passo.

[Clique aqui para acessar](#)



9) O que é o StyleSheet no React Native e como ele é usado?

O StyleSheet no React Native é uma API que permite definir estilos de forma declarativa e com melhor desempenho do que definir estilos inline. Ele oferece uma maneira eficiente de definir estilos semelhantes aos CSS, mas focados na estrutura e nos requisitos de desempenho de dispositivos móveis. Os estilos são definidos em um objeto e aplicados diretamente nos componentes.

10) Como você gerenciaria o estado em um aplicativo React Native?

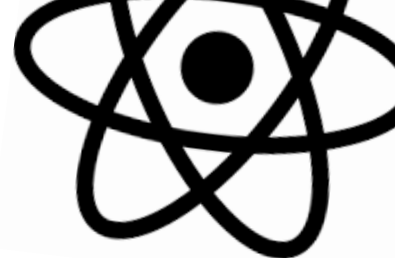
O gerenciamento de estado em React Native pode ser feito de várias maneiras:

- useState e useReducer para gerenciar estados locais em componentes funcionais.
- Context API para compartilhar estado entre vários componentes.
- Bibliotecas de gerenciamento de estado global, como Redux, MobX, ou Zustand, são comuns para projetos mais complexos, onde há necessidade de compartilhar estados de forma global e otimizada.

11) O React Native compila seu código para Android e iOS?

Não exatamente. O React Native usa uma abordagem de "compilação sob demanda" que permite que o código JavaScript seja executado em tempo de execução e transformado em elementos de interface nativos no Android ou iOS. Isso significa que o código JavaScript é interpretado em tempo real e convertido em elementos de interface nativos, o que pode levar a um desempenho inferior em comparação com o código nativo.

No entanto, o React Native também fornece a opção de compilar o código para uma versão estática, o que pode melhorar significativamente o desempenho da aplicação.



12) O que é o AsyncStorage ?

AsyncStorage é um sistema de armazenamento de chave-valor assíncrono que permite que os aplicativos armazenem dados persistentes de forma local no dispositivo do usuário em React Native. Ele permite que os aplicativos armazenem pequenos valores de dados, como preferências do usuário, dados de login, etc.

13) Para que serve fabric no React Native?

Fabric é uma arquitetura de renderização de componentes do React Native, introduzida na versão 0.62. Ela foi projetada para substituir a arquitetura anterior de renderização de componentes, conhecida como "Bridge", que às vezes pode causar atrasos na interação do usuário em dispositivos móveis com recursos limitados.

O objetivo principal do Fabric é melhorar o desempenho e a responsividade do aplicativo, permitindo que os componentes sejam renderizados de forma mais rápida e eficiente. Além disso, ele permite que o React Native seja mais escalável e mais fácil de manter a longo prazo. Isso é alcançado através da introdução de uma série de melhorias de baixo nível, como a separação dos processos de renderização do thread principal do aplicativo e a melhoria da manipulação de layout e eventos.

14) O que é o novo JSI (JavaScript Interface) ?

O JSI substitui o antigo sistema de ponte (Bridge) que comunicava o código JavaScript com o código nativo. O JSI permite uma comunicação mais direta e eficiente entre o JavaScript e o código nativo, removendo a necessidade de serializar e desserializar os dados ao passá-los entre os dois contextos, o que melhora a performance.

Full Stack PRO



Aprenda do zero a criar sites, sistemas web completos, SaaS passo a passo.

[Clique aqui para acessar](#)